

SRE Project Report

Project Members:

Ammar Hyder 22K-4816

Ausaja Hussain 22K-5186

Muhammad Abdullah 22K-5156

Date: 3rd May, 2026

**GitHub Repository of the final Reengineered Library Management
System:**

[https://github.com/MuhammadAusajaHussain/Reengineered-Library-
Management-System/](https://github.com/MuhammadAusajaHussain/Reengineered-Library-Management-System/)

Introduction

The project focuses on the **reengineering of a legacy Library Management System (LMS)** originally implemented as a console-based Java application into a modern, scalable, and maintainable software system.

- The original system was developed using object-oriented Java (NetBeans-based CLI) with tight coupling between business logic, data handling, and user interaction.
- The reengineered system transforms this into a layered architecture consisting of:
 - Domain Layer – core business entities (Book, Borrower, Loan, etc.)
 - Application Layer – business logic and use-case orchestration
 - Infrastructure Layer – database and persistence handling (Apache Derby)
 - API Layer – Spring Boot REST endpoints
 - Frontend Layer – React-based UI

This transition reflects a clear move toward **clean architecture**, improving modularity, testability, and scalability.

Selected Application

The system selected for reengineering is a **console-based JAVA Library Management System** that manages core library operations such as:

- Book catalog management
- Borrower and Staff Management
- Book issuing and returning
- Fine calculation and tracking
- Hold request handling

The original system focused on direct user interaction via terminal, in-memory data structures combined with manual database synchronization, and core entities tightly coupled with a single execution flow.

System Exploration

Overall Architecture

The original legacy system had a monolithic and tightly coupled design where business logic, data storage & user interaction were intermixed, and classes directly performed input handling, business operations (issue, return, fine calculation) and data persistence (SQL queries inside core classes).

There was no separation between layers, making system difficult to maintain, understand and extend.

Core Classes and Responsibilities

1. Library (Central Controller / God Class):

This class acts as a singleton, managing the entire system and has the majority of the business logic written in it.

It's core responsibilities include maintaining collection of persons, booksInLibrary, loans etc., along with handling book management, user creation & login, fine computation, and database interaction

2. Book Class (Overloaded Responsibilities):

This class is responsible for handling multiple responsibilities including book metadata (title, author, subject), hold request queue management, core workflows

3. Borrower Class:

This class is responsible for maintaining borrowed books (loans), hold requests, and also provides methods for updating personal info, printing borrowed/on-hold books

4. Loan Class:

This class represents transaction of issuing a book, and is responsible for handling fine calculation, fine payment interaction.

5. Staff Hierarchy (Clerk & Librarian)

These classes are inherited from Staff, which in turn extends Person class. This demonstrates a basic inheritance structure, but is not fully utilized for behavior sharing

6. Main Class (UI + Control Flow)

This acts as the entry point/means for a user to interact with the system operations. This class contains menu-driven CLI interface, and navigation logic for all operations.

Key Behavioral Workflows

1. Book Issuing Process

This process checks if book is already issued, if hold requests exists, and if borrower is eligible along with updating book status, loan records, borrower's borrowed list accordingly.

2. Hold Request Management

This workflow maintains a queue (ArrayList) of hold requests, removes expired requests using date comparison ensuring fairness (first-come-first-serve).

3. Fine Calculation

This workflow is based on return deadline, per-day fine rate and uses data difference logic. However it depends on the global state (Library class) and is mixed with user interaction

Database Interaction

The original LMS handles database interaction directly inside Library class in functions such as `makeConnection()`, `populateLibrary()`, `fillItBack()` etc.

This results in SQL logic tightly coupled with domain objects and no abstraction (no DAO/Repository pattern), resulting in high risk of errors and poor maintainability

Reverse Engineering Analysis

This phase focuses on understanding the **structure, relationships, and behavior** of the legacy LMS by analyzing its design artifacts, particularly the **class diagram** and codebase.

The goal is to extract a clear conceptual model of the system before applying reengineering.

Class Structure Overview

The legacy system follows a object-oriented model centered around a few key abstractions:

1. Person Hierarchy (Generalization):

Person acts as the base class, while the derived classes are Borrower and Staff (with Clerk and Librarian being derived from Staff class)

This demonstrates inheritance, which aligns refactoring concept of dealing with generalization, but the behavior is not fully abstracted and the type checking is still done with `getClass().getSimpleName()` indicating improper use of polymorphism

2. Core Domain Entities:

a. Book

This class has title, author, subject, and issued status as key attributes and also maintains a list of HoldRequests. It mainly interacts with Borrower, Loan, Staff

b. Borrower

This class maintains borrowed books and hold requests

c. Loan

This class represents a transactional relationship between Borrower, Book, and Staff (issuer/receiver)

d. HoldRequest

This class acts as an association class between Book and Borrower, and also includes request date

3. Central Controller: Library class

This class acts as the central class aggregating all major entities including Persons, Books, Loans. This controls system operations and database synchronization

From a reverse engineering perspective, Library behaves as a Facade (single entry point), a Controller and a Repository (improperly).

Key Relationships Identified

Inheritance

- Person → Borrower
- Person → Staff → Clerk / Librarian

Associations

- Borrower ↔ Loan ↔ Book
- Book ↔ HoldRequest ↔ Borrower

Aggregation

- Library contains collections of all major entities

Behavioral Insights

Through reverse engineering, the following behavioral characteristics were identified:

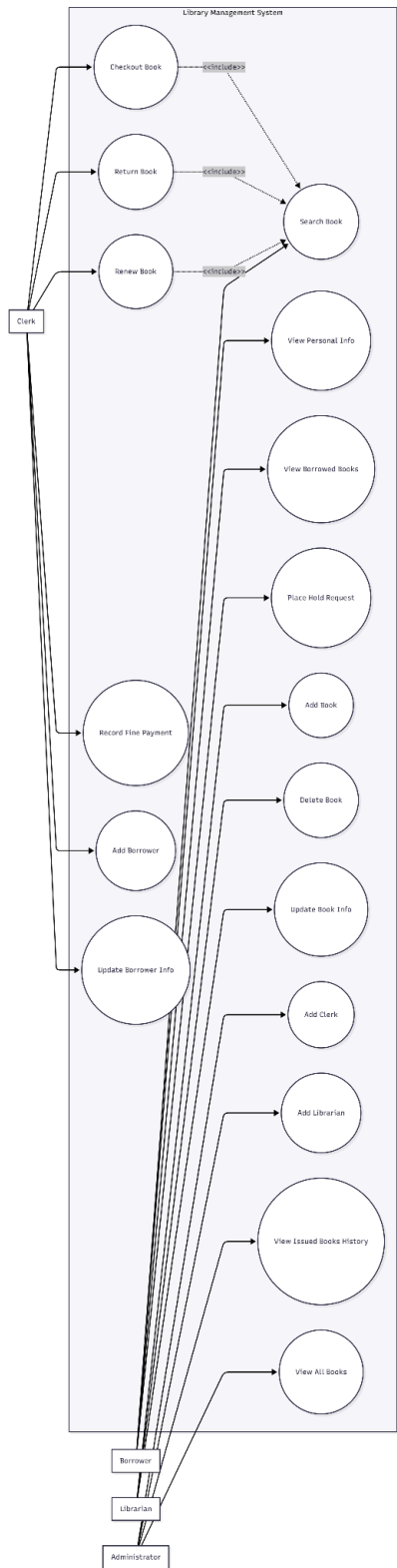
- State-driven workflows – book issuing depends on isIssued and hold queue
- Implicit business rules – FCFS hold processing; fine calculated based on delay
- Hidden dependencies – many classes rely on Library.getInstance() (global access)

Design Limitations Identified

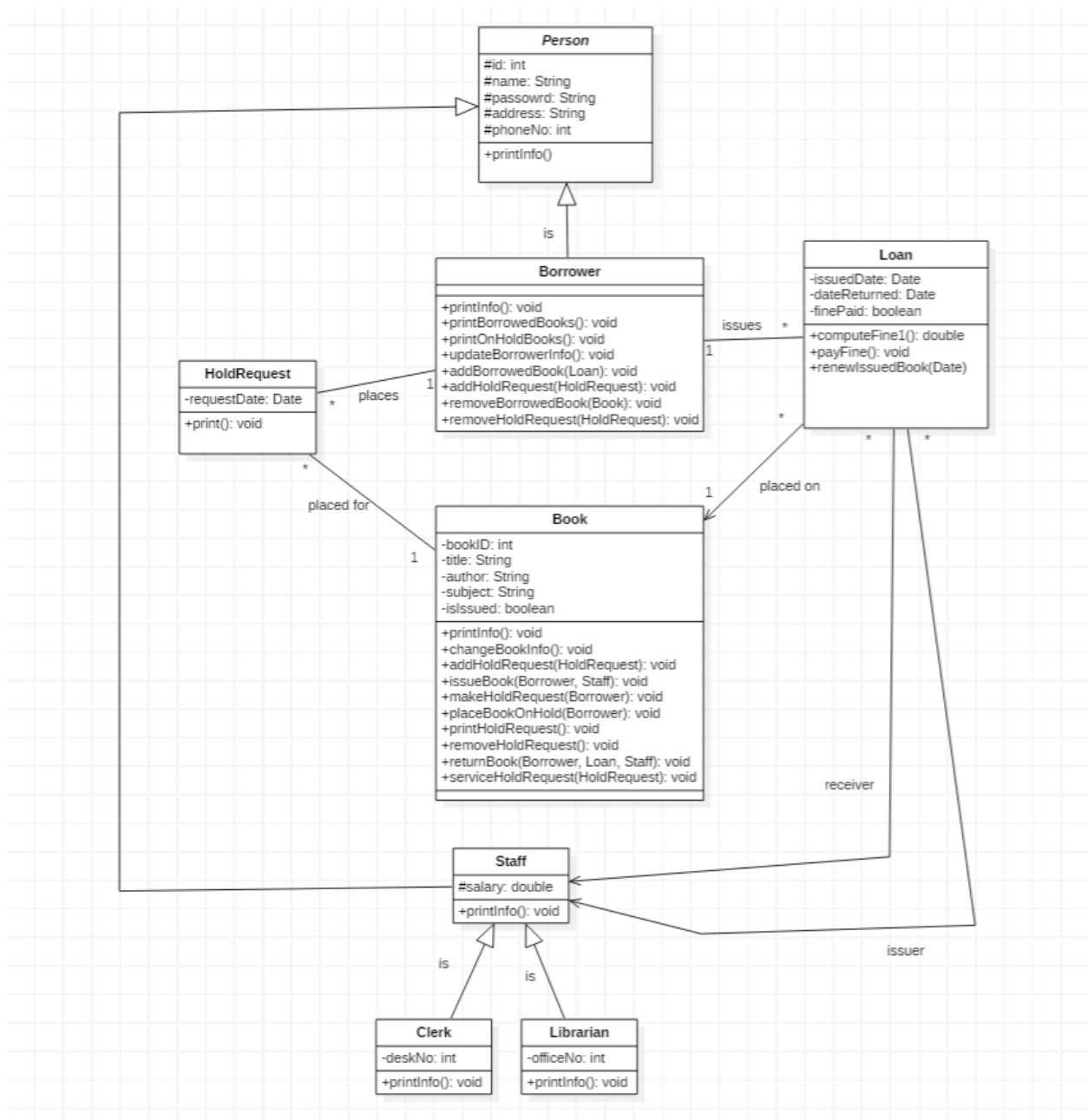
- Lack of Proper Encapsulation – Objects expose internal collections (e.g., getBorrowedBooks())
- Weak Abstraction Boundaries – Domain logic mixed with UI and persistence
- Improper Use of Inheritance – Instead of polymorphism, system uses manual type checking
- High Coupling via Singleton – Almost all classes depend on Library class

Diagrams

Use Case Diagram



Class Diagram



Weakness Identification (why was the system reengineered)

1. Monolithic Architecture

The system follows a single-layered design where UI (CLI), business logic, and database operations are tightly coupled with no clear separation between domain logic, application logic, and persistence.

This makes it difficult to modify or extend the system, and any small change risks breaking unrelated parts

2. God Class Anti-Pattern (Library Class)

The Library class performs multiple responsibilities including managing books, users and loans, handling database operations, and controlling application flow

This violates Single Responsibility Principle leading to high coupling and low cohesion, which also makes testing and debugging difficult

3. Tight Coupling Across Components

Classes directly depend on each other such as Book class depends on Borrower, Staff, and Library class, Loan class depends on Library class for fine calculation etc. There also is an extensive use of **global access** (Library.getInstance())

This reduces the system reusability making the system fragile to changes

4. Mixed Concerns (UI + Logic + Data)

The system's business logic contains input handling (Scanner, BufferedReader) and output printing (System.out.println), along with the database logic being embedded inside domain/controller classes

This violates separation of concerns, preventing reuse of logic in other interfaces (e.g., web, mobile)

5. Complex and Hard-to-Maintain Logic

The system has methods like issueBook() that contain deeply nested conditionals and multiple responsibilities (validation, processing, updating state)

This makes the system difficult to understand and debug, while also increasing the risk of introducing errors during modification

6. Improper Use of Inheritance

Although a proper hierarchy exists (Person → Staff → Clerk/Librarian), the behavior is not properly abstracted, while also using manual type checking instead of polymorphism

This limits extensibility and violates principles addressed in Dealing with Generalization

7. Inefficient Data Handling

The system uses primitive data types (e.g., phone number as integer) and direct exposure of internal collections.

This results in poor data encapsulation and an increased risk of inconsistent data states

8. Manual and Error-Prone Database Integration

The system has SQL queries embedded inside classes (such as Library class) with no proper abstraction layer (no DAO/Repository pattern). It also works on manual synchronization between memory and database

This results in the system being hard to maintain and debug with a higher chance of data inconsistency

9. Lack of Scalability and Modern Features

The system has a console-based interface that limits usability and has no support for APIs, web or distributed access, role-based access control (properly enforced)

This makes the system not suitable for real-world deployment

10. Absence of Fault Tolerance

The system has no mechanism to handle data inconsistencies and partial failures

Re-Engineering Proposal

This section describes the systematic transformation of the legacy LMS into a modern layered architecture, highlighting the applied reengineering approaches, refactoring techniques, and architectural decisions.

The goal of the reengineering process was to improve maintainability, scalability, separation of concerns, and system clarity while preserving the core library domain logic.

Selected Reengineering Approach

Primary Approach: Partial Reengineering

The system was reengineered using a Partial Reengineering Approach as the core business logic was preserved, which includes book lifecycle (issue, return, renewal), borrower management, loan tracking and hold request system.

However, the user interface was shifted from a CLI-based to web-based React UI, along with incorporating an application access layer which used REST APIs.

We also went from implementing embedded SQL logic in core classes to a structured data handling layer, changing the architecture from monolithic to a layered architecture.

Supporting Approach: Incremental Reengineering

The transformation was carried out in phases rather than opting for a full system rewrite:

- Phase 1: Identification and extraction of domain entities
- Phase 2: Separation of business logic from UI (CLI isolation)
- Phase 3: Introduction of service layer for business workflows
- Phase 4: Implementation of REST APIs using Spring Boot
- Phase 5: Integration of React frontend

A combined approach ensured that the core functionality of the system wasn't changed, while also restructuring the architecture to a layered architecture and making the system loosely coupled.

This approach reduced risk and ensured system stability, allowing continuous testing of preserved functionality.

Architectural Information

The system was restructured from a monolithic architecture into a clean, layered architecture:

- **Domain Layer** – contains pure business entities (Book, Loan, Borrower etc.) without external dependencies
- **Application Layer** – implements use case logic (Issue Book, Return Book, Renew Loan etc.) and acts as a mediator between API and domain
- **Infrastructure Layer** – handles database and its communication, while also replacing embedded SQL logic in core classes
- **API Layer** – exposes system functionality via REST endpoints and also handles request validation & response formatting
- **Presentation Layer** – React-based UI replacing console-based interaction

Refactoring Techniques Applied

The reengineering process was strongly guided by the following refactoring categories:

- Composing Methods** – large and complex methods in the legacy system were decomposed into smaller, reusable units (e.g., `issueBook()` was broken into `validateAvailability()`, `processHoldQueue()`, `createLoan()`, `updateBookStatus()`). This improved readability, reduced complexity and made unit testing easier
- Moving Features Between Objects** – responsibilities were redistributed to improved cohesion. This reduced dependency on the central core class and improved object responsibility alignment
Example: moving fine calculation logic from Library → Loan (correct domain ownership), hold request handling centralized within Book and HoldRequest model
- Organizing Data** – primitive data usage was replaced with structured objects to reduce primitive obsession (handling identity via object references rather than ID passing), and database structure was aligned with domain model. This ensured stronger data consistency, and better alignment between object model and database schema.
- Simplifying Conditional Expressions** – complex nested logic in legacy workflows were refactored. This reduced complexity, improved readability and made debugging easier.

- E. Simplifying Method Calls** – replaced generic methods like setValue(“field”, value) with explicitly domain methods like setTitle(), setSubject() etc. This improved API clarity, reduced misuse of methods, and better aligned with REST-style operations
- F. Dealing with Generalization** – removed reliance on manual type checking, encouraged polymorphic behavior, and centralized shared attributes in base class. This led to better use of inheritance principles, reduced conditional type handling and improved extensibility for future roles

Database Layer Reengineering

The original LMS had direct SQL queries inside Library class with manual synchronization between memory and database with no abstraction layer.

The reengineered LMS has introduced structured persistence layer, with database interactions isolated from business logic and clear mapping between domain entities & relational tables.

This resulted in separation of concerns between logic and storage, easier migration and database maintenance, and reduced risk of data inconsistency.

API and Frontend Transformation

The system was modernized from a terminal-based application to a distributed web application. The backend was reengineered and built in Spring Boot with role-based access control support, while the frontend was reengineered and built via React with modular screens for Books, Borrowers, Loans and Admin dashboard.

This implementation improved system usability, platform independence and is a foundation for future scalability.

Conclusion

The reengineering of the Library Management System shows how an old, difficult-to-maintain system can be improved without completely rebuilding it. The original system was tightly connected, hard to understand, and difficult to modify. By applying proper reengineering techniques, it was transformed into a system that is cleaner, more organized, and easier to work with.

One of the biggest improvements is the reduction in complexity. In the legacy system, important methods like issuing a book contained long and complicated logic with many conditions. After reengineering, this logic was broken into smaller, simpler parts using techniques like **Composing Methods** and **Simplifying Conditional Expressions**. This makes the system easier to read, debug, and maintain.

Another major improvement is how the system is structured. Previously, everything was combined in one place, especially in the **Library** class. Now, the system follows a layered structure (Domain, Application, Infrastructure, API, Frontend). This separation makes the system more flexible, so changes in one part do not affect the entire system.

The project mainly used **Partial Reengineering**, where the core logic of the system was kept, and only the necessary parts were improved or replaced. Along with this, several refactoring techniques were applied, including **Dealing with Generalization**, to improve the design and structure of the code.

Overall, the reengineered system is much easier to maintain, extend, and use. It also supports modern features like web access and APIs, which were not possible in the original system. This project clearly shows how applying proper reengineering approaches and refactoring techniques can turn a weak legacy system into a strong and modern application.